



UNIVERSIDADE REGIONAL DO NOROESTE DO
ESTADO DO RIO GRANDE DO SUL
CURSO DE CIÊNCIA DA COMPUTAÇÃO

CRISTIAN DOS SANTOS SIQUEIRA

AUTOMAÇÃO DA GERAÇÃO DE PROPOSTAS COMERCIAIS EM PDF COM DJANGO E
 $\text{\LaTeX}$

Santa Rosa/RS

2026

CRISTIAN DOS SANTOS SIQUEIRA

AUTOMAÇÃO DA GERAÇÃO DE PROPOSTAS COMERCIAIS EM PDF COM DJANGO E
L^AT_EX

Relatório de Estágio Supervisionado apresentado ao curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul — UNIJUÍ, como requisito parcial para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador(a): Prof. Manuel Osorio Binenlo

Santa Rosa/RS

2026

CRISTIAN DOS SANTOS SIQUEIRA

AUTOMAÇÃO DA GERAÇÃO DE PROPOSTAS COMERCIAIS EM PDF COM DJANGO E
L^AT_EX

Relatório de Estágio Supervisionado apresentado ao curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul — UNIJUÍ, como requisito parcial para a obtenção do grau de Bacharel em Ciência da Computação, submetido à aprovação da banca examinadora composta pelos seguintes membros:

Prof. Manuel Osorio Binelo
Orientador(a)

Prof(a). xxxx
Membro da Banca

Santa Rosa/RS, XX de XXX de XXXX

AGRADECIMENTOS

À **Lídia Linck** e ao **Nerci Linck**, pela oportunidade de realizar o estágio na BIOTECNO Indústria e Comércio Ltda., pelo ambiente de trabalho que favorece o aprendizado e pelo espaço dado para contribuir de forma técnica e significativa com os sistemas da empresa.

Ao supervisor de estágio, **Gustavo Henrique Buuron**, pela orientação direta no dia a dia do desenvolvimento, pela disponibilidade em discutir decisões técnicas e pela confiança depositada ao longo de todo o período de estágio.

Ao professor orientador, **Prof. Manuel Osorio Binelo**, pelo acompanhamento acadêmico, pelas contribuições à estruturação do trabalho e pelo suporte durante a elaboração deste relatório.

Ao **Prof. Eldair Fabricio Dornelles**, pelo desenvolvimento e disponibilização do template $\text{\LaTeX}$ utilizado neste relatório, publicado de forma aberta no GitHub e que foi fundamental para a produção deste documento com qualidade tipográfica.

RESUMO

Este relatório descreve as atividades desenvolvidas durante o estágio curricular obrigatório realizado na BIOTECNO Indústria e Comércio Ltda., empresa fabricante de equipamentos de refrigeração científica localizada em Santa Rosa, Rio Grande do Sul. O estágio foi realizado no contexto do sistema web interno da empresa, desenvolvido com o framework Django, que concentra os módulos de gestão comercial. O objetivo central do trabalho foi desenvolver um módulo de geração automatizada de propostas técnicas e comerciais em PDF, substituindo a abordagem anterior baseada na biblioteca PyLaTeX, que gerava documentos de forma estática e exigia intervenção direta no código-fonte a cada alteração de layout. A solução desenvolvida fundamenta-se em uma arquitetura orientada a layouts intercambiáveis: um novo modelo de dados, denominado LayoutOrcamento, foi projetado e implementado para permitir o cadastro e a ativação de templates de proposta diretamente pelo painel administrativo do Django, sem necessidade de alterações no código da aplicação. O template visual, elaborado por empresa de design contratada, foi codificado em LaTeX pelo estagiário e estruturado para ser processado pelo sistema de templates do Django, exigindo o tratamento adequado dos conflitos de sintaxe entre os dois sistemas. Na camada de lógica da aplicação, foram implementadas funções responsáveis pela resolução do diretório do layout ativo, pela montagem do contexto de dados, incluindo informações do cliente, itens do orçamento, descrições técnicas por conjunto de produto e condições comerciais e pela orquestração do pipeline de geração, que utiliza o compilador LuaLaTeX via latexmk para produzir o documento final. Ao término do período de estágio, o pipeline estava funcional em ambiente de desenvolvimento, com o template sendo renderizado e compilado corretamente, e a injeção das variáveis de contexto em fase de validação final antes da publicação em produção. O trabalho proporcionou a aplicação integrada de conhecimentos de Engenharia de Software, modelagem relacional, desenvolvimento web e composição tipográfica, consolidando competências técnicas e profissionais relevantes para a formação em Ciência da Computação.

Palavras-chave: django; latex; geração de documentos; proposta comercial; engenharia de software; python.

LISTA DE ILUSTRAÇÕES

Figura 1	– Estrutura e lógica do modelo DescricaoOrcamento_Conjunto	21
Figura 2	– Estrutura e lógica do modelo RegraImagem_Conjunto	22
Figura 3	– Estrutura e comportamento do modelo LayoutOrcamento	22
Figura 4	– Estrutura e lógica do template main.tex	24
Figura 5	– Estrutura e lógica da função montar_contexto_orcamento_layout . . .	25
Figura 6	– Estrutura e lógica da função orcamento_layout_pdf_view	26

LISTA DE QUADROS

Quadro 1 – Campos do modelo LayoutOrcamento	23
Quadro 2 – Comparativo entre o sistema legado e o sistema desenvolvido	27

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ANVISA	Agência Nacional de Vigilância Sanitária
CNAE	Classificação Nacional de Atividades Econômicas
CNPJ	Cadastro Nacional da Pessoa Jurídica
FDA	<i>Food and Drug Administration</i>
HTTP	<i>Hypertext Transfer Protocol</i>
MVC	<i>Model-View-Controller</i>
MTV	<i>Model-Template-View</i>
ORM	<i>Object-Relational Mapper</i>
PDF	<i>Portable Document Format</i>
SQL	<i>Structured Query Language</i>
TCC	Trabalho de Conclusão de Curso
UNIJUÍ	Universidade Regional do Noroeste do Estado do Rio Grande do Sul

SUMÁRIO

1	INTRODUÇÃO	10
1.1	JUSTIFICATIVA	10
1.2	OBJETIVOS	11
1.2.1	Objetivo Geral	11
1.2.2	Objetivos Específicos	11
1.3	ESTRUTURA DO TRABALHO	11
2	APRESENTAÇÃO DA EMPRESA	13
2.1	HISTÓRICO	13
2.2	MISSÃO, VISÃO E VALORES	13
3	METODOLOGIA	15
4	REVISÃO BIBLIOGRÁFICA	16
4.1	DJANGO E O PADRÃO MTV	16
4.2	LATEX E GERAÇÃO DE DOCUMENTOS	16
4.3	MODELAGEM RELACIONAL E DJANGO ORM	17
4.4	ARQUITETURA DE SOFTWARE E BOAS PRÁTICAS	17
5	DESENVOLVIMENTO	19
5.1	ANÁLISE E LEVANTAMENTO DE REQUISITOS	19
5.2	MODELAGEM	20
5.2.1	Contexto dos Modelos Existentes	20
5.2.1.1	Modelo DescricaoOrcamento_Conjunto	20
5.2.1.2	Modelo RegraImagem_Conjunto	21
5.2.2	Novo Modelo: LayoutOrcamento	22
5.3	IMPLEMENTAÇÃO	23
5.3.1	Codificação do Template LaTeX	23
5.3.2	Funções da <i>View</i>	24
5.3.2.1	Função montar_contexto_orcamento_layout	25
5.3.2.2	Função orcamento_layout_pdf_view	26
5.3.3	Integração com o Django Admin	26
5.4	RESULTADOS E DISCUSSÕES	27
6	CONSIDERAÇÕES FINAIS	28
6.1	CONCLUSÕES	28
6.2	TRABALHOS FUTUROS	29

REFERÊNCIAS	30
ANEXOS	31
ANEXO A – TEMPLATE DE PROPOSTA COMERCIAL	32

1 INTRODUÇÃO

A transformação digital tem impactado de forma crescente os processos internos das empresas industriais, tornando a automação de documentos e a integração entre sistemas operacionais cada vez mais estratégicas para a competitividade. Nesse contexto, o desenvolvimento de software empresarial, em especial sistemas de gestão customizados, emerge como uma disciplina central da Engenharia de Software, exigindo dos profissionais habilidades que vão desde a modelagem de dados até a geração automatizada de documentos com alto padrão visual.

A BIOTECNO Indústria e Comércio Ltda., empresa fabricante de equipamentos de refrigeração científica localizada em Santa Rosa (RS), opera com um sistema web interno denominado Bioweb, desenvolvido em Django. Dentro do módulo comercial do Bioweb, a geração de propostas técnicas e comerciais é uma das atividades de maior impacto direto no relacionamento com clientes, pois o documento produzido representa a empresa perante o mercado.

Até o início deste estágio, os orçamentos em PDF eram gerados por meio da biblioteca PyLaTeX, que construía o documento de forma programática e estática, ou seja, qualquer alteração no layout da proposta exigia modificação direta no código-fonte Python. Esse modelo limitava a capacidade de atualização visual das propostas e criava dependência técnica para ajustes que, em essência, são decisões de negócio e identidade visual. Paralelamente, a empresa contratou uma empresa de design para a criação de um novo layout de proposta comercial, o qual precisava ser integrado ao sistema existente de forma flexível e sustentável.

Este relatório descreve as atividades desenvolvidas ao longo do estágio curricular obrigatório realizado na BIOTECNO, com foco na concepção e implementação de um sistema de geração automatizada de propostas comerciais em PDF. O trabalho envolveu a codificação do template LaTeX a partir do layout entregue pela empresa de design, a modelagem de um novo conjunto de entidades no banco de dados, e o desenvolvimento da lógica de geração de documentos na camada de *views* do Django com ênfase em uma arquitetura que permita a troca de layouts diretamente pelo painel administrativo, sem necessidade de alterações no código.

1.1 JUSTIFICATIVA

A prática de estágio supervisionado constitui etapa fundamental na formação do profissional de Ciência da Computação, pois permite a aplicação concreta dos conhecimentos construídos ao longo do curso em problemas reais de complexidade e escala incompatíveis com o ambiente acadêmico isolado. A exposição a decisões de arquitetura de software com impacto direto em processos de negócio desenvolve no estagiário uma visão sistêmica que complementa a formação teórica.

Do ponto de vista acadêmico, o projeto demandou a aplicação integrada de conteúdos de Engenharia de Software (modelagem de requisitos, padrões de projeto), Banco de Dados (modelagem relacional, migrações), Desenvolvimento Web (arquitetura MVC/MTV, *views* baseadas em função) e Sistemas de Composição Tipográfica (L^AT_EX), demonstrando a interdisciplinaridade inerente ao desenvolvimento de sistemas reais. A necessidade de integrar dois sistemas com gramáticas conflitantes — o motor de templates do Django e a sintaxe do L^AT_EX — e de tomar decisões arquiteturais com impacto direto na manutenibilidade do sistema são exemplos de desafios que a formação acadêmica prepara teoricamente, mas que só se consolidam no contato com um ambiente produtivo real.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver um módulo de geração automatizada de propostas técnicas e comerciais em PDF, integrado ao sistema Django da BIOTECNO, com suporte a layouts intercambiáveis gerenciáveis pelo painel administrativo, substituindo a abordagem estática anterior baseada na biblioteca PyLaTeX.

1.2.2 Objetivos Específicos

- a) Analisar o processo vigente de geração de orçamentos e levantar os requisitos funcionais e não funcionais do novo sistema;
- b) Traduzir o layout visual elaborado pela empresa de design para um template L^AT_EX funcional, parametrizável via sistema de templates do Django;
- c) Modelar e implementar o modelo LayoutOrçamento no banco de dados, incluindo suporte ao upload de templates compactados e ao mecanismo de layout ativo único;
- d) Desenvolver as funções de resolução de diretório de layout, montagem de contexto e geração de PDF na camada de *views* do Django, integrando o pipeline completo do banco de dados à entrega do documento ao navegador;
- e) Documentar o processo de desenvolvimento e os procedimentos necessários para manutenção futura da solução.

1.3 ESTRUTURA DO TRABALHO

Para melhor compreensão, este relatório está organizado em capítulos sequenciais. Além desta Introdução, o Capítulo 2 apresenta o perfil da BIOTECNO Indústria e Comércio Ltda. e o ambiente tecnológico em que o estágio foi realizado. O Capítulo 3 descreve a sistemática de trabalho adotada, as ferramentas utilizadas e a forma como as demandas foram gerenciadas ao longo do período. O Capítulo 4 fundamenta tecnicamente o projeto, discutindo os principais

conceitos e tecnologias empregados. O Capítulo 5 detalha o processo completo de análise, modelagem e implementação do sistema de geração de orçamentos. Por fim, o Capítulo 6 apresenta as conclusões e perspectivas de trabalhos futuros.

2 APRESENTAÇÃO DA EMPRESA

A BIOTECNO Indústria e Comércio Ltda. é uma empresa brasileira fabricante de refrigeradores e freezers científicos, sediada em Santa Rosa, no noroeste do estado do Rio Grande do Sul. Com atuação voltada ao mercado de saúde e pesquisa científica, a empresa desenvolve soluções para a conservação de imunobiológicos, hemoderivados, medicamentos termolábeis e amostras laboratoriais, atendendo a um público exigente quanto à precisão técnica e à conformidade regulatória. Este capítulo apresenta o histórico da organização, seus pilares estratégicos e a estrutura tecnológica em que o estágio foi realizado.

2.1 HISTÓRICO

A BIOTECNO foi fundada em 22 de maio de 2001, em Santa Rosa (RS), por um grupo de profissionais com conhecimento técnico na área de refrigeração e uma visão clara das lacunas existentes no mercado nacional de conservação de materiais biológicos. Desde os seus primeiros anos de operação, a empresa orientou sua atuação ao desenvolvimento de câmaras científicas voltadas à cadeia de frio, segmento que exige rigorosa estabilidade térmica e conformidade com normas sanitárias nacionais e internacionais.

Um dos marcos que consolidou a empresa no mercado nacional foi o desenvolvimento de uma linha de câmaras para conservação de imunobiológicos, termolábeis e hemoderivados com capacidade de manter operação durante períodos de ausência de energia elétrica, inovação inédita no país à época e que atendeu a uma demanda crítica de postos de saúde, clínicas e hospitais localizados em regiões com fornecimento elétrico instável. Essa inovação fez da BIOTECNO a marca de refrigeração científica mais premiada do Brasil, com reconhecimento também em feiras e eventos internacionais do setor.

Com mais de duas décadas de trajetória, a empresa consolidou sua posição como referência nacional em refrigeração médico-científica, com equipamentos registrados no Ministério da Saúde e certificados pela *U.S. Food and Drug Administration* (FDA), ampliando sua presença para o mercado internacional. Atualmente, a BIOTECNO conta com mais de 20 anos de história desenvolvendo soluções para a cadeia de frio, mantendo sua sede na Estrada Municipal João Hermes, 915, bairro Glória, em Santa Rosa (RS), e uma filial ativa na mesma cidade.

2.2 MISSÃO, VISÃO E VALORES

Os pilares estratégicos da BIOTECNO refletem sua origem técnica e sua vocação para o mercado de saúde. A missão da empresa é desenvolver e fabricar refrigeradores e freezers científicos de alta qualidade, contribuindo para a conservação segura de materiais biológicos e para a proteção da saúde humana, sintetizada em seu lema institucional *Conserving Life*.

A visão é ser reconhecida como a principal referência brasileira em refrigeração científica, com presença consolidada nos mercados nacional e internacional e portfólio alinhado às mais exigentes normas regulatórias de saúde do mundo.

Os valores que orientam as práticas da organização incluem: inovação contínua, com investimento permanente em pesquisa e desenvolvimento de produtos; qualidade técnica, expressa pela conformidade com as normas da ANVISA e da FDA; proximidade com o cliente, com compromisso declarado de total satisfação e disponibilização de diversos canais de atendimento; e responsabilidade com a saúde pública, reconhecendo que os equipamentos fabricados integram diretamente a infraestrutura de saúde de hospitais, farmácias, postos de vacinação e centros de pesquisa.

3 METODOLOGIA

O estágio foi realizado de forma presencial nas dependências da BIOTECNO, com carga horária distribuída ao longo da semana conforme acordo com o supervisor Gustavo Henrique Buuron. As demandas de desenvolvimento foram conduzidas de maneira iterativa e incremental, próxima a uma abordagem ágil informal: as tarefas eram discutidas com o supervisor, executadas em ciclos curtos e revisadas à medida que os resultados parciais emergiam, permitindo ajustes contínuos de escopo e implementação.

A organização das atividades seguiu um fluxo composto pelas seguintes etapas: (1) compreensão do problema ou requisito apresentado pelo supervisor; (2) análise do sistema existente, leitura de *models*, *views* e templates já implementados, para entender o contexto de integração; (3) implementação da solução, com testes manuais no ambiente de desenvolvimento; e (4) revisão conjunta com o supervisor antes de integrar a funcionalidade à base de código principal.

As ferramentas e tecnologias centrais do projeto foram:

- a) **Python e Django**: linguagem e framework base do sistema web da empresa, utilizados em todas as etapas de desenvolvimento (Django Software Foundation, 2024);
- b) **LaTeX** com compilador **LuaLaTeX** via `latexmk`: sistema de composição tipográfica utilizado para a geração dos documentos PDF (Collins, 2023);
- c) **PostgreSQL**: sistema gerenciador de banco de dados relacional utilizado pela aplicação;
- d) **Git**: controle de versão para rastreamento das alterações no código-fonte;
- e) **Django Admin**: interface administrativa nativa do Django, utilizada como ponto de gerenciamento dos layouts de orçamento.

A comunicação técnica com o supervisor ocorreu diariamente, permitindo que decisões de arquitetura como a escolha de resolver o layout ativo em tempo de geração em vez de vinculá-lo ao modelo `Orçamento` fossem validadas antes de serem implementadas. Essa dinâmica reduziu retrabalho e garantiu que a solução fosse coerente com as necessidades operacionais da empresa.

4 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta os fundamentos teóricos e tecnológicos que embasam o trabalho desenvolvido durante o estágio. São abordados os principais conceitos relacionados ao desenvolvimento web com Django, à geração de documentos com LaTeX e à modelagem de dados relacional, pilares sobre os quais a solução de geração automatizada de propostas comerciais foi construída.

4.1 DJANGO E O PADRÃO MTV

Django é um *framework* web de alto nível escrito em Python, projetado para facilitar o desenvolvimento rápido de aplicações web seguras e de fácil manutenção (Django Software Foundation, 2024). Sua arquitetura segue o padrão MTV (*Model-Template-View*), uma variação do amplamente conhecido MVC (*Model-View-Controller*), no qual o *Model* define a estrutura dos dados e sua relação com o banco de dados; o *Template* é responsável pela apresentação, seja HTML, seja um documento LaTeX, como utilizado neste projeto; e a *View* concentra a lógica de negócio, recebendo requisições HTTP, consultando os modelos e retornando uma resposta ao cliente (Holovaty e Kaplan-Moss, 2009).

Um dos componentes centrais do Django utilizado neste projeto é o ORM (*Object-Relational Mapper*), que permite interagir com o banco de dados por meio de classes Python, abstraindo a escrita de SQL. Cada modelo Django é uma subclasse de `django.db.models.Model`, e seus atributos correspondem às colunas de uma tabela relacional. As migrações, geradas automaticamente pelo comando `makemigrations`, traduzem alterações nos modelos em operações de banco de dados reproduzíveis e controladas por versão (Django Software Foundation, 2024).

O sistema de *templates* do Django suporta uma linguagem própria de marcação que permite interpolação de variáveis, estruturas de controle (*if*, *for*) e herança de templates. Por padrão, os delimitadores dessa linguagem são chaves duplas para variáveis e blocos percentual-chave para *tags*, o que pode gerar conflitos com a sintaxe do LaTeX, cujo caractere de percentual tem função de comentário e cujas chaves são usadas extensivamente como delimitadores de comandos. Esse conflito demanda atenção durante o desenvolvimento de templates LaTeX processados pelo Django.

4.2 LATEX E GERAÇÃO DE DOCUMENTOS

LaTeX é um sistema de preparação de documentos baseado no programa de composição tipográfica TeX, desenvolvido por Donald Knuth (Knuth, 1984). Diferentemente de editores WYSIWYG (*What You See Is What You Get*), no LaTeX o autor descreve a estrutura e o conteúdo

do documento por meio de comandos textuais, e o sistema se encarrega da diagramação final, resultando em documentos com qualidade tipográfica superior (Lamport, 1994).

Para a compilação de documentos LaTeX em arquivos PDF, este projeto utiliza o compilador **LuaLaTeX**, variante moderna do LaTeX com suporte nativo a Unicode e a fontes OpenType, em conjunto com o utilitário `latexmk`, que automatiza o processo de compilação, executando o compilador quantas vezes forem necessárias para resolver referências cruzadas, sumários e outros elementos que dependem de múltiplas passagens (Collins, 2023).

A abordagem adotada neste projeto, renderizar um template `.tex` com o sistema de templates do Django e, em seguida, compilá-lo com `latexmk`, é uma estratégia consolidada para geração de PDFs com alto controle tipográfico em aplicações web, especialmente quando o documento exige layout complexo, como tabelas posicionadas com precisão, imagens integradas e fontes customizadas.

4.3 MODELAGEM RELACIONAL E DJANGO ORM

A modelagem relacional é a base sobre a qual os sistemas de informação organizam e persistem seus dados. No contexto do Django, a definição dos modelos determina diretamente a estrutura do banco de dados e, conseqüentemente, as possibilidades de consulta e relacionamento entre entidades (Elmasri e Navathe, 2011).

Neste projeto, um aspecto relevante da modelagem é a decisão de **não** vincular o layout de orçamento diretamente ao modelo `Orçamento` por meio de uma chave estrangeira. Em vez disso, o sistema resolve o layout ativo em tempo de geração, consultando o modelo `LayoutOrçamento` pelo campo `is_active`. Essa decisão arquitetural segue o princípio de baixo acoplamento (*loose coupling*): o modelo de negócio central (`Orçamento`) permanece desacoplado das decisões de apresentação (layout), facilitando a evolução independente de ambos (Martin, 2002).

O campo `AutoSlugField`, fornecido pela biblioteca `django-autoslug`, gera automaticamente um identificador textual único a partir de outro campo do modelo — neste caso, a descrição do layout — sem requerer intervenção manual, o que simplifica a identificação de registros em rotas e na interface administrativa.

4.4 ARQUITETURA DE SOFTWARE E BOAS PRÁTICAS

O princípio da separação de responsabilidades (*Separation of Concerns*) orienta a organização do código em camadas com papéis bem definidos (Dijkstra, 1982). No contexto deste projeto, ele se manifesta na divisão entre: (a) a camada de modelos, responsável pela persistência e pela lógica de dados; (b) a camada de *views*, responsável pela orquestração, ou seja, a resolução do layout ativo, montagem do contexto e acionamento do compilador; e (c) a camada de templates, responsável exclusivamente pela estrutura visual do documento LaTeX.

Essa separação garante que alterações no layout não exijam modificações na lógica da *view*, e que mudanças nas regras de negócio não impactem a estrutura do template, reduzindo o custo de manutenção e aumentando a testabilidade do sistema (Martin, 2002).

5 DESENVOLVIMENTO

Este capítulo descreve o processo de análise, modelagem e implementação do módulo de geração automatizada de propostas técnicas e comerciais (orçamentos) em PDF. O trabalho partiu do diagnóstico do sistema legado, passou pela concepção de uma nova arquitetura orientada a layouts intercambiáveis e chegou à implementação de um pipeline completo, do banco de dados à entrega do arquivo PDF ao navegador.

5.1 ANÁLISE E LEVANTAMENTO DE REQUISITOS

O processo de levantamento de requisitos foi conduzido por meio de reuniões com o supervisor de estágio e da análise direta do sistema existente. A inspeção do código legado revelou que a geração de orçamentos era realizada pela função `generate_orcamento`, localizada no módulo `comercial/latex.py`, que utilizava a biblioteca PyLaTeX para construir o documento PDF programaticamente, adicionando tabelas, seções e formatações por meio de chamadas de função Python. Embora funcional, essa abordagem apresentava limitações estruturais relevantes:

- a) O layout do documento estava **codificado diretamente no código Python**, impossibilitando alterações visuais sem intervenção de desenvolvedor;
- b) Não havia suporte a **múltiplos layouts**, qualquer nova identidade visual exigia a reescrita da função de geração;
- c) A inclusão de **descrições técnicas dos equipamentos** por conjunto (componente de produto) não era contemplada de forma sistemática, prejudicando a completude da proposta.

Simultaneamente, a empresa contratou uma empresa de design para elaborar um novo modelo visual de proposta comercial. Esse layout foi entregue como referência visual (não como arquivo LaTeX), cabendo ao estagiário a sua codificação no formato `.tex`.

A partir dessa análise, foram definidos os seguintes requisitos:

Requisitos Funcionais:

- a) O sistema deve permitir o cadastro de layouts de orçamento via painel administrativo do Django, com upload do template compactado em formato `.zip`;
- b) Apenas um layout pode estar ativo por vez; a ativação de um novo layout deve desativar automaticamente o anterior;
- c) A geração do PDF deve utilizar o layout ativo no momento da requisição, sem necessidade de configuração manual;

- d) O documento gerado deve incluir: informações cadastrais do cliente, tabela de equipamentos com valores, descrições técnicas por conjunto de produto, imagens dos equipamentos e condições comerciais;
- e) O sistema deve ser compatível com orçamentos contendo múltiplos equipamentos, cada um com sua própria composição de conjuntos e opcionais.

Requisitos Não Funcionais:

- a) A solução deve ser implementada dentro do *framework* Django, seguindo os padrões já estabelecidos no sistema da empresa;
- b) O template LaTeX deve ser processado pelo sistema de templates do Django, exigindo tratamento adequado dos conflitos de sintaxe entre os dois sistemas;
- c) O compilador utilizado deve ser o LuaLaTeX via `latexmk`, padrão já adotado em outros módulos do sistema;
- d) A troca de layout não deve exigir alterações no código-fonte da aplicação.

5.2 MODELAGEM

5.2.1 Contexto dos Modelos Existentes

O sistema comercial já possuía os seguintes modelos centrais para o processo de orçamento:

- a) `Orcamento`: representa a proposta comercial, com número de série, cliente, vendedor, versão e status;
- b) `OrcamentoEquipamento`: representa cada item do orçamento, armazenando o campo `equipamentodetail` — uma cadeia de códigos de conjuntos de produto separados por ponto, que define a composição técnica do equipamento;
- c) `OrcamentoCondicoes`: armazena as condições comerciais da proposta (prazo, frete, garantia, pagamento);
- d) `DescricaoOrcamento_Conjunto`: associa conjuntos de produto a textos descritivos utilizados nas propostas, com suporte a prioridade e configuração de chave;
- e) `RegraImagem_Conjunto`: associa conjuntos de produto a imagens representativas, utilizadas no documento.

Os dois últimos modelos são detalhados nas subseções a seguir, pois receberam extensões diretas durante o desenvolvimento.

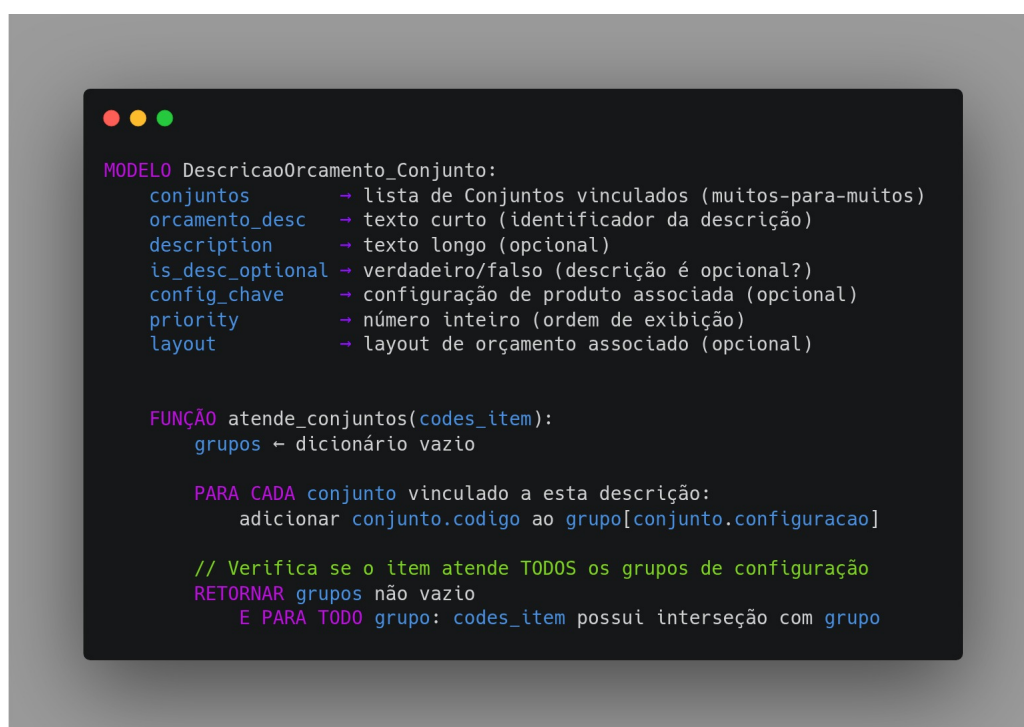
5.2.1.1 Modelo `DescricaoOrcamento_Conjunto`

O modelo `DescricaoOrcamento_Conjunto` já existia no sistema e é responsável por vincular textos descritivos a conjuntos de produto. Cada registro associa um ou mais conjuntos

(relação muitos-para-muitos) a um identificador curto (`orcamento_desc`), um texto longo opcional (`description`), uma configuração de chave (`config_chave`) e um campo de prioridade para ordenação.

No contexto deste projeto, duas contribuições foram realizadas sobre esse modelo: a adição de uma chave estrangeira opcional para `LayoutOrcamento`, permitindo vincular descrições a um layout específico; e a implementação do método `atende_conjuntos(codes_item)`, que verifica se o registro é aplicável a um dado equipamento, agrupa os conjuntos vinculados por configuração e retorna verdadeiro somente se `codes_item` possui interseção com todos os grupos, garantindo que apenas descrições pertinentes ao equipamento sejam incluídas na proposta. A Figura 1 ilustra a estrutura e a lógica do modelo.

Figura 1 – Estrutura e lógica do modelo `DescricaoOrcamento_Conjunto`



Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

5.2.1.2 Modelo RegraImagem_Conjunto

O modelo `RegraImagem_Conjunto` segue estrutura análoga ao anterior, já existindo anteriormente, porém voltado a imagens representativas dos equipamentos. Cada registro vincula um ou mais conjuntos ao arquivo de imagem (`image`), a um identificador (`orcamento_desc`) e a uma URL opcional (`link`). O método `atende_imagem(codes_item)` aplica a mesma lógica de interseção por grupos de configuração para determinar se a imagem é pertinente ao equipamento em questão. A Figura 4 apresenta a estrutura do modelo.

Figura 2 – Estrutura e lógica do modelo RegraImagem_Conjunto

```

MODELO RegraImagem_Conjunto:
  conjuntos → lista de Conjuntos vinculados (muitos-para-muitos)
  orcamento_desc → texto curto (identificador da imagem)
  image → arquivo de imagem
  link → URL associada (opcional)

FUNÇÃO atende_imagem(codes_item):
  grupos ← dicionário vazio

  PARA CADA conjunto vinculado a esta regra:
    adicionar conjunto.codigo ao grupo[conjunto.configuracao]

  RETORNAR grupos não vazio
  E PARA TODO grupo: codes_item possui interseção com grupo

```

Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

5.2.2 Novo Modelo: LayoutOrcamento

Para suportar layouts intercambiáveis, foi projetado e implementado o modelo LayoutOrcamento, cuja estrutura é apresentada na Figura 3 e detalhada no Quadro 1.

Figura 3 – Estrutura e comportamento do modelo LayoutOrcamento

```

MODELO LayoutOrcamento:
  description → texto (nome do layout)
  slug → identificador único gerado automaticamente a partir do nome
  fonte → arquivo ZIP com templates LaTeX (opcional)
  is_active → verdadeiro/falso (layout em uso?)

AO SALVAR:
  SE este layout está sendo ativado (is_active = verdadeiro):
    desativar TODOS os outros layouts do banco de dados

  salvar normalmente

```

Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

Uma decisão central de modelagem foi **não incluir** uma chave estrangeira de LayoutOrcamento no modelo Orcamento. Essa escolha mantém o modelo de negócio desacoplado das decisões de apresentação: o layout ativo é resolvido em tempo de geração, por meio de uma consulta direta ao registro marcado como ativo, e não é persistido no orçamento. Isso significa que diferentes gerações do mesmo orçamento podem utilizar layouts distintos, conforme o layout ativo no momento — comportamento adequado à realidade operacional da empresa, onde a troca de layout é uma decisão de identidade visual global, não individual por proposta.

Quadro 1 – Campos do modelo LayoutOrcamento

Campo	Tipo	Descrição
description	CharField	Nome descritivo do layout
slug	AutoSlugField	Identificador único gerado a partir de description
fonte	FileField	Arquivo .zip contendo o template LaTeX e seus assets
is_active	BooleanField	Indica se este é o layout em uso

Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

Os modelos `DescricaoOrcamento_Conjunto` e `RegraImagem_Conjunto` receberam chaves estrangeiras opcionais para `LayoutOrcamento`, permitindo que descrições e imagens sejam vinculadas a um layout específico. O sistema aplica um filtro com fallback: retorna registros do layout ativo ou, na ausência, registros sem layout definido — garantindo compatibilidade com dados cadastrados antes da implementação do novo sistema.

O mecanismo de layout único ativo é garantido pelo método `save()` sobrescrito no modelo: ao ativar um layout, todos os demais são automaticamente desativados antes da persistência do registro, conforme ilustrado na Figura 3.

5.3 IMPLEMENTAÇÃO

A implementação foi dividida em três frentes interdependentes: (a) a codificação do template LaTeX; (b) o desenvolvimento das funções de suporte na *view*; e (c) a integração do pipeline completo de geração de PDF.

5.3.1 Codificação do Template LaTeX

O layout visual elaborado pela empresa de design foi traduzido para um arquivo `main.tex` estruturado para ser processado pelo sistema de templates do Django. Esse processo exigiu atenção especial ao **conflito de delimitadores** entre os dois sistemas: o LaTeX utiliza extensivamente o caractere de percentual como início de comentário e chaves como delimitadores de argumentos de comandos, enquanto o Django utiliza chaves duplas para variáveis e blocos percentual-chave para *tags* de controle.

A solução adotada, já consolidada em outros módulos do sistema, como `viewsinspecao.py`, consiste em configurar o motor de templates com delimitadores customizados, substituindo a sintaxe padrão do Django por sequências que não colidem com a gramática do LaTeX. Essa abordagem permite que o arquivo `.tex` seja ao mesmo tempo um template Django válido e um documento LaTeX sintaticamente correto.

O template é organizado em seções que correspondem às partes visuais do documento:

- a) **Cabeçalho:** logotipo da empresa e identificação da proposta;

- b) **Informações cadastrais:** dados do cliente (razão social, CNPJ, município, contato);
- c) **Tabela de equipamentos:** itens do orçamento com quantidade, valor unitário e valor total;
- d) **Descrições técnicas:** para cada equipamento, as descrições dos conjuntos que o compõem são listadas em formato de tabela com identificadores de atributo e valores;
- e) **Imagens dos equipamentos:** imagens associadas aos conjuntos via `RegraImagem_Conjunto`;
- f) **Condições de fornecimento:** pagamento, frete, garantia, prazo e serviços inclusos;
- g) **Rodapé:** dados de contato e assinatura do consultor.

Figura 4 – Estrutura e lógica do template `main.tex`



Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

5.3.2 Funções da View

Na camada de *views*, foram implementadas três funções principais que compõem o pipeline de geração, descritas a seguir.

5.3.2.1 Função montar_contexto_orcamento_layout

A função `montar_contexto_orcamento_layout` é responsável por construir o dicionário de dados que será injetado no template LaTeX. Recebe o orçamento e a versão como parâmetros e, internamente, resolve o layout ativo, as condições comerciais e os equipamentos vinculados ao orçamento. Para cada equipamento, os códigos de conjuntos são extraídos do campo `equipamentodetail` e utilizados para filtrar as descrições e imagens aplicáveis, por meio dos métodos `atende_conjuntos` e `atende_imagem` descritos anteriormente. As descrições são pré-carregadas com filtro de fallback — layout ativo ou sem layout definido — e ordenadas por prioridade. O contexto retornado contém todos os dados necessários para a renderização completa do documento, sem lógica de apresentação embutida. A Figura 5 apresenta o pseudocódigo da função.

Figura 5 – Estrutura e lógica da função `montar_contexto_orcamento_layout`

```

FUNÇÃO montar_contexto_orcamento_layout(orcamento, versao):
    versao      ← versao ou versão atual do orçamento
    condicoes   ← buscar condições comerciais do orçamento/versão
    layout      ← buscar layout ativo no banco de dados

    equipamentos ← buscar equipamentos do orçamento/versão

    // Pré-carregar descrições do layout ativo OU sem layout definido
    descricoes_base ← DescricaoOrcamento_Conjunto filtradas por
                      (layout = layout ativo OU layout = nulo)
                      ordenadas por prioridade

    imagens_base ← todas as RegraImagem_Conjunto

    itens ← lista vazia
    total ← 0

    PARA CADA equipamento (com índice):
        subtotal ← quantidade × valor unitário
        total    ← total + subtotal
        codes_item ← conjuntos do equipamento

        descricoes ← descricoes_base que atendem codes_item
        imagens     ← imagens_base que atendem codes_item

        adicionar à itens:
            posição, item, descrição, quantidade,
            valor unitário, subtotal, descrições, imagens, opcionais

    RETORNAR:
        orcamento, cliente, contato, condições,
        equipamentos, itens, layout, total, versão

```

Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

5.3.2.2 Função `orcamento_layout_pdf_view`

A função `orcamento_layout_pdf_view` é a *view* principal, responsável por orquestrar todo o pipeline de geração. Recebe a requisição HTTP, o identificador do orçamento e a versão, e executa a seguinte sequência: localiza o orçamento no banco de dados; resolve o layout ativo e o diretório de trabalho do template; monta o contexto de dados; renderiza o template `main.tex` com as variáveis substituídas; salva o arquivo `.tex` renderizado em disco; aciona o compilador LuaLaTeX via `latexmk`; e retorna o PDF gerado como resposta HTTP. Cada etapa possui tratamento de erro individual: ausência de layout ativo, arquivo ZIP não encontrado, template ausente ou falha na compilação resultam em resposta 404 com registro do erro no terminal. A Figura 6 apresenta o pseudocódigo da função.

Figura 6 – Estrutura e lógica da função `orcamento_layout_pdf_view`

```

FUNÇÃO orcamento_layout_pdf_view(requisicao, pk, versao):
    orcamento ← buscar orçamento pelo pk ou erro 404

    layout ← buscar layout ativo no banco de dados
    SE layout não encontrado:
        RETORNAR erro 404

    // Extrair templates do ZIP para pasta de trabalho
    layout_dir ← resolver diretório do layout
    SE arquivo ZIP não encontrado:
        RETORNAR erro 404

    SE main.tex não existe no diretório:
        RETORNAR erro 404

    // Montar dados e renderizar template
    contexto ← montar_contexto_orcamento_layout(orcamento, versao)
    arquivo_tex ← renderizar template LaTeX com o contexto
    salvar arquivo_tex em disco

    // Compilar PDF
    TENTAR:
        EXECUTAR compilador LaTeX (latexmk) sobre arquivo_tex
    SE compilação falhar:
        exibir log de erro no terminal
        RETORNAR erro 404

    SE arquivo PDF não foi gerado:
        RETORNAR erro 404

    RETORNAR arquivo PDF como resposta

```

Fonte: Própria do autor (2026), elaborado com Carbon (carbon.now.sh).

5.3.3 Integração com o Django Admin

O modelo `LayoutOrcamento` foi registrado no painel administrativo do Django, permitindo que gestores realizem o upload de novos templates (`.zip`) e ativem um layout diretamente

pela interface web, sem qualquer interação com o código-fonte. Ao marcar o campo `is_active` como verdadeiro em um registro, o método `save()` sobrescrito desativa automaticamente todos os demais layouts antes de salvar, garantindo a consistência do estado ativo único.

5.4 RESULTADOS E DISCUSSÕES

Ao final do período de estágio, o pipeline de geração de orçamentos com o novo layout estava funcional em ambiente de desenvolvimento: o template LaTeX era renderizado corretamente pelo sistema de templates do Django e compilado com sucesso pelo `latexmk`, produzindo o PDF com o layout definido pela empresa de design. A injeção das variáveis de contexto, dados do cliente, itens, descrições técnicas e condições comerciais, estava em fase de validação final antes da integração ao ambiente de produção.

O Quadro 2 apresenta uma comparação entre o sistema legado (PyLaTeX) e a nova arquitetura implementada.

Quadro 2 – Comparativo entre o sistema legado e o sistema desenvolvido

Aspecto	Sistema Legado (PyLaTeX)	Novo Sistema (Django + LaTeX)
Alteração de layout	Requer modificação de código Python	Realizada via painel administrativo
Suporte a múltiplos layouts	Não	Sim, com ativação dinâmica
Descrições técnicas por conjunto	Não estruturado	Gerenciadas via modelo dedicado
Qualidade tipográfica	Limitada pela abstração da biblioteca PyLaTeX	Total controle via LaTeX nativo
Separação de responsabilidades	Baixa (layout no código)	Alta (layout no template)

Fonte: Própria do autor (2026).

A decisão arquitetural de resolver o layout em tempo de geração — em vez de persistir a referência no modelo `Orçamento` — mostrou-se adequada: manteve o modelo de negócio limpo e permitiu que a troca de layout afete todas as gerações futuras sem exigir alterações em registros históricos. A principal ressalva é que orçamentos gerados em momentos diferentes poderão ter layouts distintos, o que é aceitável no contexto operacional da empresa e pode ser tratado futuramente com o armazenamento do PDF gerado associado ao registro do orçamento.

O tratamento do conflito de delimitadores entre Django e LaTeX, validado pela referência ao padrão já utilizado no módulo de inspeção (`viewsinspecao.py`), demonstrou que a solução é extensível a qualquer template LaTeX processado pelo sistema, consolidando um padrão técnico reaproveitável por toda a aplicação.

6 CONSIDERAÇÕES FINAIS

O estágio realizado na BIOTECNO Indústria e Comércio Ltda. teve como objetivo central o desenvolvimento de um módulo de geração automatizada de propostas técnicas e comerciais em PDF, com suporte a layouts intercambiáveis gerenciáveis pelo painel administrativo do Django. O objetivo foi alcançado: ao término do período, o sistema estava implementado e funcional em ambiente de desenvolvimento, com pipeline de geração completo da resolução do layout ativo à entrega do PDF compilado pelo LuaLaTeX, pronto para ser integrado ao ambiente de produção na próxima atualização da aplicação.

6.1 CONCLUSÕES

A experiência de estágio proporcionou uma imersão em decisões técnicas que raramente se apresentam com a mesma complexidade e consequência no ambiente acadêmico. A necessidade de integrar dois sistemas com gramáticas conflitantes, o motor de templates do Django e a sintaxe do LaTeX, exigiu compreensão profunda de como cada camada funciona internamente, indo além do uso superficial das ferramentas. Da mesma forma, a decisão de não acoplar o layout ao modelo Orcamento foi fruto de uma análise de *trade-offs* reais: entre consistência histórica e flexibilidade operacional, optou-se pela flexibilidade — decisão validada pelo contexto de negócio da empresa.

Do ponto de vista técnico, o estágio consolidou conhecimentos em Django ORM, modelagem relacional, geração de documentos com LaTeX e integração de processos externos via subprocess, conteúdos abordados no curso de Ciência da Computação e que, no contexto prático, revelaram nuances e interdependências não evidentes na teoria. A leitura e compreensão de um sistema de software já em produção, com múltiplos módulos, convenções próprias e histórico de decisões acumulado, foi por si só um aprendizado valioso para a formação profissional.

Do ponto de vista comportamental, a dinâmica de trabalho com ciclos curtos de revisão junto ao supervisor reforçou a importância da comunicação técnica clara e da validação incremental de decisões de arquitetura. Erros foram identificados e corrigidos precocemente, evitando retrabalho de maior escala.

Um aspecto relevante que o estágio evidenciou, e que raramente é abordado no ambiente acadêmico, é a prática de substituição progressiva de código legado por soluções mais modernas e padronizadas. A migração do módulo de orçamentos do PyLaTeX para a arquitetura de templates Django não foi motivada por uma falha crítica do sistema anterior, mas pela necessidade de padronização interna e pela oportunidade de tornar o sistema mais sustentável a longo prazo. Identificar esse tipo de demanda, propor uma solução arquitetural coerente e executá-la sem quebrar o que já funcionava é uma habilidade que a formação acadêmica prepara teoricamente,

mas que só se desenvolve plenamente na prática.

Por fim, o fato de já atuar na empresa de forma remunerada antes do início do estágio curricular teve impacto direto na produtividade e na qualidade das entregas. O conhecimento prévio do sistema Bioweb, das convenções de código adotadas pela equipe e da estrutura do projeto eliminou a curva de ambientação inicial, permitindo que o desenvolvimento começasse com foco imediato no problema técnico. Essa familiaridade também facilitou a leitura de módulos existentes como referência, como o uso do `viewsinspecao.py` para resolver o conflito de delimitadores Django e $\text{\LaTeX}$, acelerando decisões que, em outro contexto, exigiriam investigação mais prolongada.

6.2 TRABALHOS FUTUROS

A continuidade natural do trabalho desenvolvido aponta para os seguintes encaminhamentos:

- **Publicação em produção:** integrar o novo sistema de geração ao ambiente de produção da aplicação, substituindo definitivamente a função legada baseada em PyLaTeX, após validação completa das variáveis de contexto com dados reais;
- **Múltiplos layouts por tipo de venda:** ampliar o modelo `LayoutOrçamento` para suportar diferentes layouts por categoria de venda (direta, licitação, revenda), permitindo que a seleção do template seja automatizada com base nos atributos do orçamento;
- **Interface de pré-visualização:** desenvolver uma funcionalidade de pré-visualização do PDF no painel administrativo antes da ativação de um novo layout, reduzindo o risco de publicação de templates com erros de formatação.

REFERÊNCIAS

COLLINS, J. *Latexmk: Fully Automated L^AT_EX Document Generation*. [S.l.], 2023. Disponível em: <<https://ctan.org/pkg/latexmk>>. Citado 2 vezes nas páginas 15 e 17.

DIJKSTRA, E. W. On the role of scientific thought. *Selected Writings on Computing: A Personal Perspective*, Springer-Verlag, New York, p. 60–66, 1982. Citado na página 17.

Django Software Foundation. *Django: The Web Framework for Perfectionists with Deadlines*. 2024. <<https://www.djangoproject.com>>. Acesso em: 10 abr. 2025. Citado 2 vezes nas páginas 15 e 16.

ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 6. ed. São Paulo: Pearson Addison-Wesley, 2011. ISBN 9788579360855. Citado na página 17.

HOLOVATY, A.; KAPLAN-MOSS, J. *The Definitive Guide to Django: Web Development Done Right*. 2. ed. New York: Apress, 2009. ISBN 9781430219361. Citado na página 16.

KNUTH, D. E. *The T_EXbook*. Reading, Massachusetts: Addison-Wesley, 1984. ISBN 9780201134476. Citado na página 16.

LAMPORT, L. *L^AT_EX: A Document Preparation System*. 2. ed. Reading, Massachusetts: Addison-Wesley, 1994. ISBN 9780201529838. Citado na página 17.

MARTIN, R. C. *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, New Jersey: Pearson Education, 2002. ISBN 9780135974445. Citado 2 vezes nas páginas 17 e 18.

Anexos

ANEXO A – TEMPLATE DE PROPOSTA COMERCIAL

A seguir, apresenta-se o template de proposta técnica e comercial gerado pelo sistema desenvolvido durante o estágio, com o layout produzido pela empresa de design e codificado em $\text{\LaTeX}$ pelo autor.

PROPOSTA TÉCNICA/COMERCIAL

BT-10100/420 TOP



Biotecno Indústria e Comércio Ltda.
CNPJ 04.470.103/0001-76

Matriz - Santa Rosa - RS / Estrada Municipal João Hermes, 915 - Lajeado Ipê, Bairro Glória | Brasil | CEP 98785-810

Filial 1 - Unidade Pós-vendas - Santa Rosa - RS / Rua Pirapó, 613 - Bairro Timbaúva / CEP 98781-054

Filial 2 - Centro de Distribuição e Assistência Técnica - Embu das Artes - SP / Av. Jorge Alfredo Camasmie, 474 - Bairro Gramado - Galpão J4/D1 / CEP 06816-050

Contato: +55 55 3513-0686 | vendas@biotecno.com.br | www.biotecno.com.br



Seja bem-vindo à Biotecno

Há mais de 25 anos desenvolvemos soluções para a cadeia de frio em aplicações que não admitem falhas.

Nossos equipamentos são projetados para garantir segurança, rastreabilidade e conformidade regulatória, atendendo às exigências da Anvisa e aos padrões internacionais FDA (U.S. Food & Drug Administration). É com satisfação que lhe apresentamos a solução ideal para a sua necessidade.



Onde estamos

Matriz em Santa Rosa – RS, operando em duas plantas;
Unidade em São Paulo dedicada à distribuição de equipamentos e assistência técnica;
Atuação em todo o território nacional por meio da nossa Rede de Assistência Técnica.

A Biotecno está presente onde segurança e conformidade são indispensáveis.

Grandes marcas confiam no nosso trabalho



Levamos para o mundo a tecnologia que produzimos



Somos reconhecidos por meio de prêmios e certificações, nacionais e internacionais



Rede de Assistência Técnica

Nossa relação com o cliente não termina na entrega do equipamento — ela começa ali. Contamos com uma rede nacional de assistências técnicas autorizadas e profissionais continuamente capacitados para garantir o pleno funcionamento dos equipamentos Biotecno:

- Manual de Política de Assistência Técnica disponível para consulta;
- Mais de 50 assistentes técnicos autorizados em todo o Brasil;
- Treinamento técnico contínuo e certificação especializada;
- Reposição de peças testadas e rastreadas;
- Atendimento remoto ágil e especializado.

Quando você precisar, estaremos ao seu lado. Isso é parceria.

Treinamento Assistentes Técnicos 2025



A seguir, apresentamos nossa proposta técnica e comercial elaborada especialmente para atender à sua solicitação.

Informações cadastrais

Cliente: Unimed Pres Prud Coop De Trabalho Medico
E-mail: compras@unimedprudente.com.br
Cidade/UF: Presidente Prudente, SP
CNPJ: 44863959000398
IE:

Telefone/WhatsApp:
Material armazenado:
Faixa de trabalho:
Contato: Jane
Função:

Informações do(s) equipamento(s)

Item	Modelo/Descrição	Quantidade	Valor Unit.	Valor Total
1	BT - 1100/420 TOP	1	R\$24.640,00	R\$24.640,00
Total Equipamentos				R\$24.640,00

Condições de fornecimento

Pagamento	1 + 3 Boleto (mediante aprovação de crédito)
Impostos	ICMS (Incluso nos preços. Alíquota variável de acordo com o estado.) + IPI (0%)
Informações adicionais de pagamento	À vista com 3.5% de desconto ou cartão de crédito até 10 vezes
Validade da proposta	30 Dias
Previsão de embarque	30 dias após a confirmação da compra. (CONFORME DEMANDA DE ESTOQUE OU NECESSIDADE DO CLIENTE).
Garantia	12 Meses
Serviços ofertados	Instalação e Treinamento Online / Processos de Qualificação QP - QI - QO - QD (Conf. ABNT/NBR/ISO 17025/17)
Frete	FOB

Descrição do(s) equipamento(s)



Gabinete Externo	Construído em chapa de aço eletrogalvanizado com película de inox escovado, possui rodízios giratórios traváveis.
Gabinete Interno	Em chapa de aço inoxidável, apresenta três (3) gavetas com corredeiras para um deslizamento suave.
Capacidade em Litros	Possui capacidade de ** litros.
Isolamento Térmico	Em poliuretano, com espessura de 75MM.
Homogeneização	Sistema de ar forçado com desligamento automático ao abrir a porta.
Porta/Tampa	De vidro duplo com sistema antiembaçante.
Refrigeração	Opera com compressor hermético e gás ecológico isento de CFC, contando com sistema de degelo seco.
Painel de Comando	Painel frontal soft-touch e display LCD com leitura simultânea das temperaturas.
Alarmes	Alertas para temperatura fora da faixa, falta de energia e porta aberta, com tecla de inibição do alarme.
Faixa de Trabalho	Entre 2°C e 8°C, com ponto fixo em 4°C
Software de Gerenciamento	Gratuito para emissão de relatórios e gráficos, permitindo o gerenciamento via internet.
Iluminação Interna	Iluminação em LED com acionamento automático ao abrir a porta.
Datalogger	Realiza o registro automático de temperaturas e eventos do equipamento ao longo do tempo.
Saída USB/Pen Drive	Para exportação de relatórios e gráficos de desempenho, inclusive retroativos, com dados criptografados.
Registrador de Temperatura	Registrador eletrônico de temperaturas mínimas e máximas, com visualização direta no display.
Certificação de Software	Software validável pelas normas RDC 658/2022, RDC 430/2022 e CFR 21 Part 11.
Sistema de Segurança	Termostato de segurança instalado em paralelo.
Discadora Fixa	Sistema automático que realiza ligações via central telefônica para até seis (6) números.
Sistema de Emergência	Com Conversor 12V para 110/220V que mantém todas as funções por até 24 horas.
Sistema de Backup	Bateria selada para suporte às funções do painel por até 48 horas.
Nível de Ruído	** dB
Dimensões Externas	** alt ** larg
Dimensões Internas	** alt ** larg
Funcionamento Bivolt	127 ou 220 VOLTS 50/60Hz (através de chave seletora)

Agradecemos seu interesse em nossa linha de equipamentos e soluções para cadeia de frio.

Essa proposta é o primeiro passo de uma parceira forte e duradoura.

Cristian dos Santos Siquiera